PINOUT — R080 Timer MEGA

Documento estratto direttamente dal codice sorgente. Ogni riga riporta il riferimento `file:linea` per tracciabilità. Target: **Arduino Mega 2560 (ATmega2560)**.

---

1. Mappa pin → componente

Pin Arduino	Direzione	Componente	Costante / Oggetto	Riferimento
D1 (TX0)	OUT (inutilizzato come RX di SoftwareSerial)	7-seg display (RX placeholder)	`SoftwareSerial(1, A2)`	`include/variables.h:51`
D2	IN_PULLUP	Pulsante principale	`MAIN_BTN_PIN` → `OneButton mainButton`	`include/constants.h:11`, `include/variables.h:60`, `include/functions/config.h:23`
D3	IN	Encoder modalità — DT	`ENC_MODE_DT_PIN` → `Encoder encoderMode`	`include/constants.h:18`, `include/variables.h:54`
D4	IN	Encoder modalità — CLK	`ENC_MODE_CLK_PIN` → `Encoder encoderMode`	`include/constants.h:19`, `include/variables.h:54`
D16	IN	Encoder precisione — SW (button)	`ENC_PRECISION_SW_PIN` → `EncoderButton encoderPrecisionButton`	`include/constants.h:24`, `include/variables.h:58`
D17	IN	Encoder modalità — SW (button)	`ENC_MODE_SW_PIN` → `EncoderButton encoderModeButton`	`include/constants.h:20`, `include/variables.h:57`
D18	IN	Encoder precisione — DT	`ENC_PRECISION_DT_PIN` → `Encoder encoderPrecision`	`include/constants.h:22`, `include/variables.h:55`
D19	IN	Encoder precisione — CLK	`ENC_PRECISION_CLK_PIN` → `Encoder encoderPrecision`	`include/constants.h:23`, `include/variables.h:55`
D20 (SDA)	I²C	LCD Waveshare RGB	hardware I²C (HW Wire)	`src/main.cpp:5`, `include/variables.h:52`
D21 (SCL)	I²C	LCD Waveshare RGB	hardware I²C (HW Wire)	`src/main.cpp:5`, `include/variables.h:52`
D27	IN	Keypad 4×4 — riga 4	`MATRIX_ROWS_PIN[3]`	`include/constants.h:40`
D29	IN	Keypad 4×4 — riga 3	`MATRIX_ROWS_PIN[2]`	`include/constants.h:40`
D31	IN	Keypad 4×4 — riga 2	`MATRIX_ROWS_PIN[1]`	`include/constants.h:40`
D33	IN	Keypad 4×4 — riga 1	`MATRIX_ROWS_PIN[0]`	`include/constants.h:40`
D35	OUT	Keypad 4×4 — colonna 4	`MATRIX_COLS_PIN[3]`	`include/constants.h:39`
D37	OUT	Keypad 4×4 — colonna 3	`MATRIX_COLS_PIN[2]`	`include/constants.h:39`

Pin Arduino	Direzione	Componente	Costante / Oggetto	Riferimento
D39	OUT	Keypad 4×4 — colonna 2	`MATRIX_COLS_PIN[1]`	`include/constants.h:39`
D41	OUT	Keypad 4×4 — colonna 1	`MATRIX_COLS_PIN[0]`	`include/constants.h:39`
D47	OUT	Relè ingranditore	`ENLARGER_RELAY_PIN` (PORTL bit PL2)	`include/constants.h:10`, `include/functions/config.h:21-22`, `include/functions/timer_isr.h:36-39`
D49	OUT	Buzzer	`BUZZER_PIN` (PWM via `tone()`)	`include/constants.h:9`, `include/functions/config.h:20`, `include/functions/sound.h:42`
A2	OUT (TX di SoftwareSerial)	7-seg display (DIN)	`DISPLAY_7_SEG_PIN` → `SoftwareSerial sevenSegmentsDisplay`	`include/constants.h:12`, `include/variables.h:51`, `include/functions/config.h:59`

---

2. Componente → pin (vista per dispositivo)

Pulsante principale (OneButton)

- **Pin:** D2, `INPUT_PULLUP`
- **Logica:** active-low (chiusura a GND)
- **Oggetto:** `OneButton mainButton(MAIN_BTN_PIN, true)` (`true` = activeLow + pullup) — `include/variables.h:60`
- **Config:** `pinMode(MAIN_BTN_PIN, INPUT_PULLUP)` — `include/functions/config.h:23`

Relè ingranditore

- **Pin Arduino:** D47
- **Port AVR:** `PORTL` bit `PL2` (ATmega2560 port mapping)
- **Direzione:** `OUTPUT`, init `LOW` — `include/functions/config.h:21-22`, `src/main.cpp:28`
- **Macro di flip del pin** — `include/functions/timer_isr.h:36-39`:
  - `RELAY_ON()` → `PORTL |= (1 << PL2)` — `include/functions/timer_isr.h:38`
  - `RELAY_OFF()` → `PORTL &= ~(1 << PL2)` — `include/functions/timer_isr.h:39`
  - **Nota (dal commento nel codice, righe 27-35):** `PORTL` è in *extended I/O* (indirizzo `0x10B`, fuori dal range `0x00..0x1F`), quindi GCC **non** emette `SBI`/`CBI` ma una sequenza `lds`/`andi` (o `ori`)/`sts` **non atomica**. È comunque sicura: dentro l'ISR perché `SREG.I=0` disabilita le altre interrupt; dal main perché nessun'altra ISR scrive `PL2` (l'unico altro bit di `PORTL` toccato è `PL0`, buzzer via `tone()` → race solo audio, irrilevante).
- **Gate per canale:** `relayStart()` controlla `channelsRelayActivated[channel-1]` prima di flippare — `include/functions/base.h:15-19`
- **Chiusura automatica fine countdown:** ISR esegue `RELAY_OFF()` quando `g_remainingMs` raggiunge 0, guardato da `g_relayGate` — `include/functions/timer_isr.h:73-75`

Buzzer

- **Pin:** D49
- **Direzione:** `OUTPUT` — `include/functions/config.h:20`
- **Pilotaggio:** `tone(BUZZER_PIN, freq, duration)` da `emitSound()` / `tickSound()` — `include/functions/sound.h:42, 56, 77`
- **Frequenze:** `TONE_UP = 600 Hz`, `TONE_DOWN = 300 Hz` — `include/constants.h:42-43`

Encoder "modalità" (sinistro)

- **Tipo:** rotativo + pulsante
- **Pin:**
  - DT = D3 (`ENC_MODE_DT_PIN`)
  - CLK = D4 (`ENC_MODE_CLK_PIN`)
  - SW = D17 (`ENC_MODE_SW_PIN`)
- **Oggetti:** `Encoder encoderMode(D3, D4)` + `EncoderButton encoderModeButton(D17)` — `include/variables.h:54, 57`
- Riferimento: `include/constants.h:18-20`

Encoder "precisione" (destro)

- **Tipo:** rotativo + pulsante
- **Pin:**
  - DT = D18 (`ENC_PRECISION_DT_PIN`)
  - CLK = D19 (`ENC_PRECISION_CLK_PIN`)
  - SW = D16 (`ENC_PRECISION_SW_PIN`)
- **Oggetti:** `Encoder encoderPrecision(D18, D19)` + `EncoderButton encoderPrecisionButton(D16)` — `include/variables.h:55, 58`
- Riferimento: `include/constants.h:22-24`

LCD Waveshare RGB 16×2 (I²C)

- **Bus:** I²C hardware del Mega 2560
  - SDA = D20
  - SCL = D21
- **Driver:** `Waveshare_LCD1602_RGB lcd(16, 2)` — `include/variables.h:52`
- **Init:** `lcd.init()` + `lcd.setRGB(255, 0, 0)` (retroilluminazione rossa fissa) — `include/functions/config.h:43-44`
- **Caratteri custom:**
  - slot 1 = ♪ (`note[8]`) — `include/variables.h:18`, `include/functions/config.h:45`
  - slot 2 = ⚡ (`spina[8]`) — `include/variables.h:19`, `include/functions/config.h:46`
- **Note:** la costante `LCD_ADDRESS = 0x27` (`include/constants.h:14`) è vestigiale — il driver Waveshare ha indirizzi I²C cablati internamente (LCD `0x3E`, backlight RGB `0x60`) e ignora questa costante.

Display 7-segmenti seriale (SparkFun-style)

- **Pin Arduino:** A2 (usato come digital output)
- **Trasporto:** `SoftwareSerial sevenSegmentsDisplay(1, A2)` — RX=D1 (non utilizzato), TX=A2 — `include/variables.h:51`

- **Baud rate:** 9600 — `include/functions/config.h:59`
- **Comandi serie scritti come byte raw:**
  - `0x76` reset — `include/functions/config.h:60`, `include/functions/config.h:68`
  - `0x7A <byte>` set brightness (0..255) — `include/functions/config.h:61-62`
  - `0x77 0b00000100` punto decimale dopo la 1ª cifra (formato `XX.X`) — `include/functions/config.h:70-71`
  - 4 caratteri ASCII = 4 cifre

Keypad 4×4

- **Layout dei tasti** (`MATRIX_KEYS`, `include/constants.h:29-34`):

```
1  2  3  A
4  5  6  B
7  8  9  C
*  0  #  D
```

- **Pin colonne:** D41, D39, D37, D35 (`MATRIX_COLS_PIN`) — `include/constants.h:39`
- **Pin righe:** D33, D31, D29, D27 (`MATRIX_ROWS_PIN`) — `include/constants.h:40`
- **Driver:** libreria `chris--a/Keypad`. Istanziato con righe e colonne **scambiati** rispetto alla dichiarazione di `MATRIX_KEYS` (i due swap si annullano):
  - `Keypad(makeKeymap(MATRIX_KEYS), MATRIX_ROWS_PIN, MATRIX_COLS_PIN, MATRIX_ROWS_N, MATRIX_COLS_N)` — `include/variables.h:50`
- **Debounce:** 50 ms — `include/functions/config.h:36`

---

3. Risorse hardware AVR utilizzate

Risorsa MCU	Uso	Riferimento
Timer1 (16-bit)	CTC, ISR a 1 kHz, motore temporale unico del firmware	`include/functions/timer_isr.h:54-94`
Timer2 (8-bit)	usato dalla `tone()` di Arduino per il buzzer	`include/functions/sound.h:42`
Timer0 (8-bit)	usato dal core Arduino per `millis()` / `delay()` (solo `delay(3000)` dello splash in `setup()`; il firmware non usa `millis()` nella logica)	`src/main.cpp` (`setup()`)
PORTL bit PL2	flip diretto del relè (anche da ISR) — vedi nota su extended I/O e non-atomicità	`include/functions/timer_isr.h:36-39`
EEPROM (4 KB)	persistenza 13 byte, magic `0xA5` (vedi `functions/storage.h`)	`src/main.cpp:3, 37`
TWI (I²C)	bus LCD su SDA=D20, SCL=D21	`src/main.cpp:5`
USART0	`Serial` 9600 baud, solo banner debug `=== R080 TIMER PRONTO ===`	`src/main.cpp:27, 55`

Configurazione Timer1 (sintesi)

- `OCR1A = 249` → 250 tick × 4 µs (prescaler 64 @ 16 MHz) = **1 ms per ISR**
- `WGM12 = 1` → modalità CTC (TOP = OCR1A)
- `CS11 | CS10` → prescaler 64
- `OCIE1A` → abilita interrupt `Compare Match A`
- Riferimento: `include/functions/timer_isr.h:54-64`

---

4. Pin non utilizzati / liberi

Sul Mega 2560 i pin digital sono D0..D53 e analogici A0..A15. Dal codice attuale risultano **occupati**:

`D1, D2, D3, D4, D16, D17, D18, D19, D20 (SDA), D21 (SCL), D27, D29, D31, D33, D35, D37, D39, D41, D47, D49, A2`.

Pin **liberi** (non referenziati nel sorgente):

- Digital: D0, D5–D15, D22–D26, D28, D30, D32, D34, D36, D38, D40, D42–D46, D48, D50–D53
- Analog: A0, A1, A3–A15

> Nota: D0/D1 sono condivisi con `Serial` (USART0). D1 è dichiarato come pin RX del costruttore `SoftwareSerial(1, A2)` ma il 7-seg è solo TX, quindi D1 non è realmente in uso dal SoftwareSerial — può essere considerato utilizzabile con cautela.

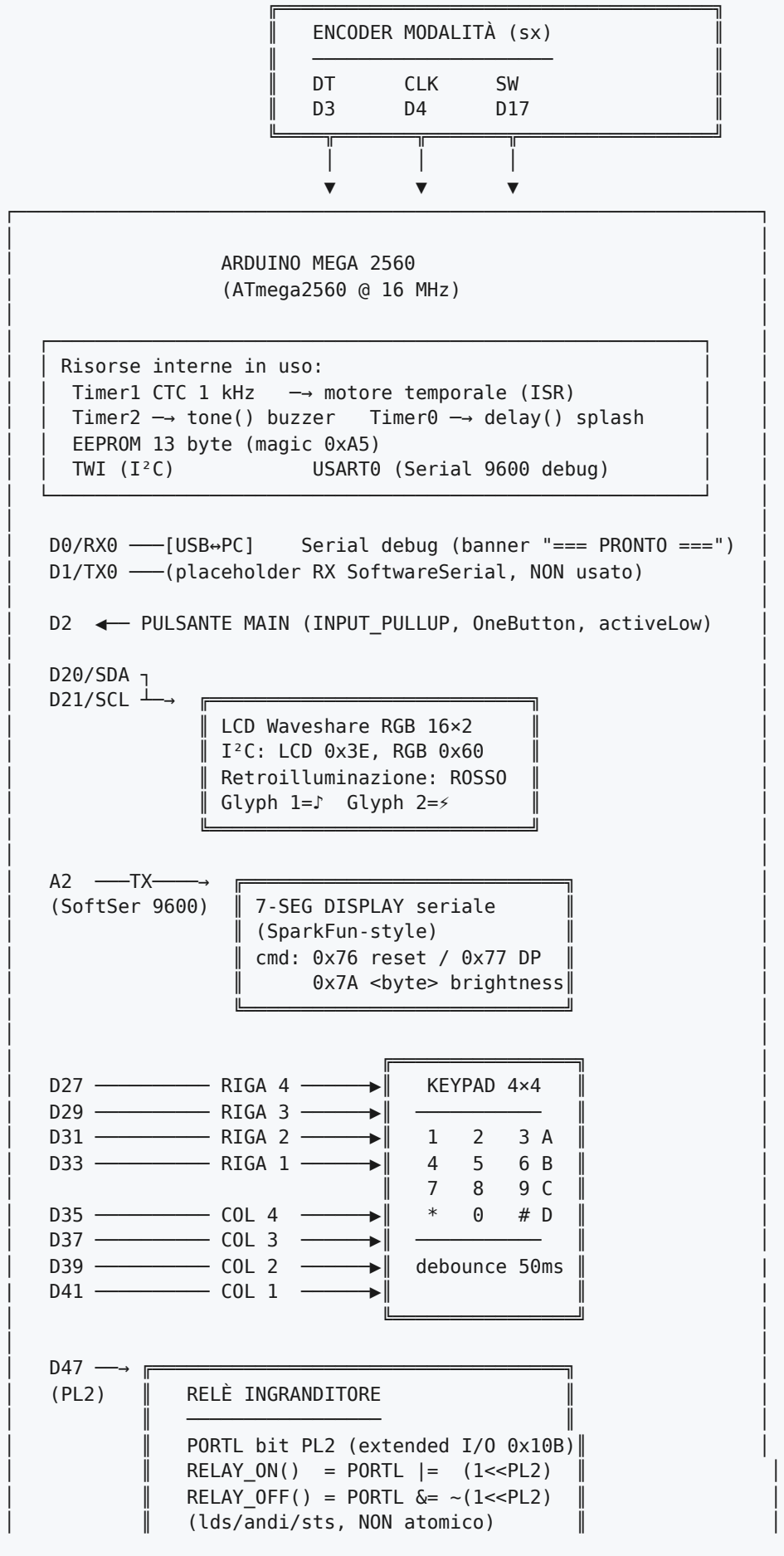
---

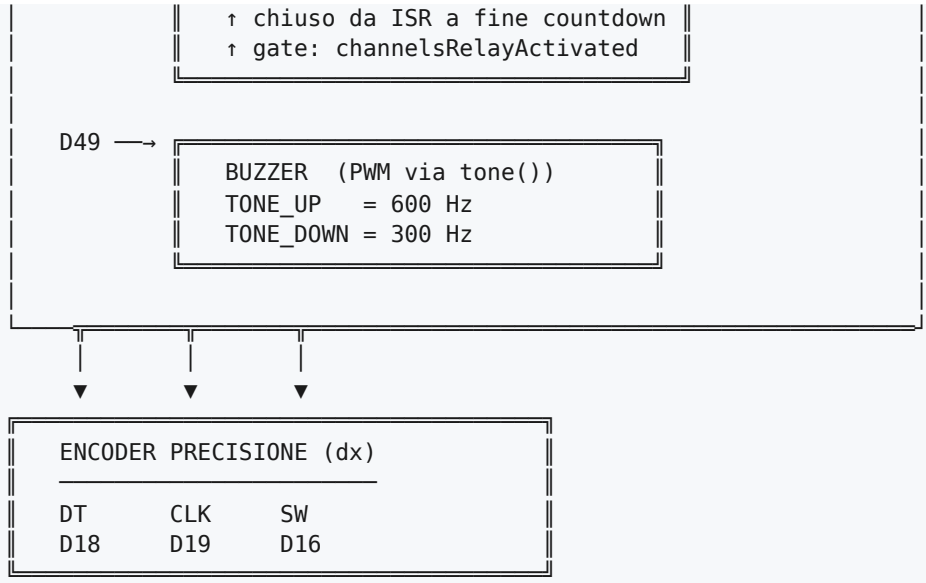
5. File sorgente coinvolti

File	Ruolo nella pinout
`include/constants.h`	dichiarazione di **tutte** le costanti pin + tabella tasti
`include/variables.h`	istanziazione degli oggetti hardware (Keypad, Encoder, EncoderButton, OneButton, SoftwareSerial, Waveshare_LCD1602_RGB)
`include/functions/config.h`	`initializePinMode()`, init keypad/LCD/7-seg
`include/functions/timer_isr.h`	macro `RELAY_ON()` / `RELAY_OFF()`, setup Timer1
`include/functions/base.h`	`relayStart()` con gate per canale
`include/functions/sound.h`	uso di `tone()` / `noTone()` sul buzzer
`src/main.cpp`	unico `.cpp`, `setup()` chiama init in ordine

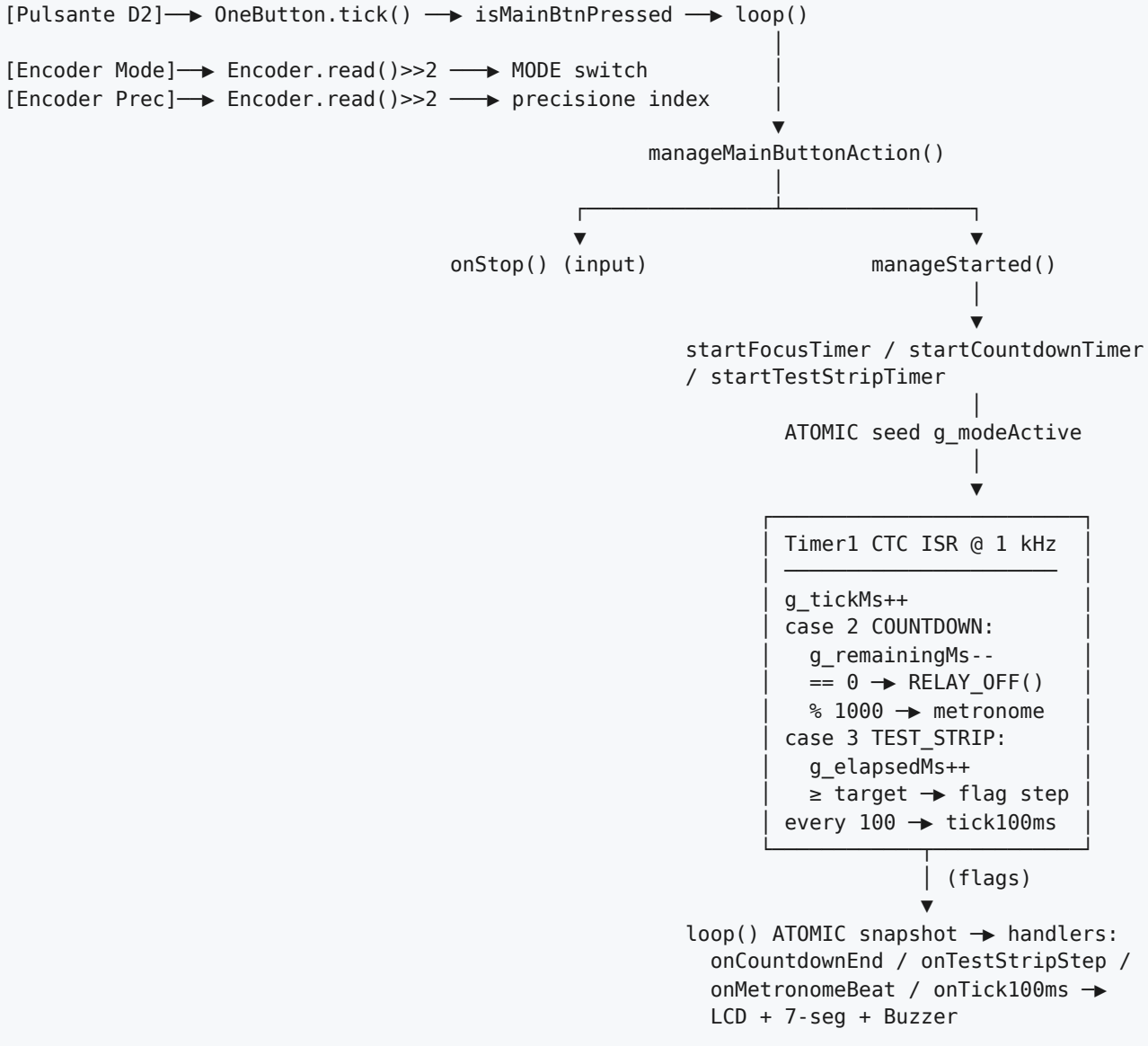
---

6. Schema ASCII — vista perimetrale





7. Schema ASCII — flusso segnali



Legenda

Simbolo	Significato
`─→`	senso del flusso del segnale
`←──`	direzione opposta (input verso MCU)
`╔═══╗`	bordo dispositivo / blocco logico
`(PL2)`	port AVR sottostante al pin Arduino